

Experiment No 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

A Smart Contract is a self-executing program stored on the blockchain that automatically enforces the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries, ensuring transparency, security, and trust between parties. Smart contracts are written in Solidity (for Ethereum-based systems) and executed on the Ethereum Virtual Machine (EVM).

Significance of Smart Contracts in the Ethereum Blockchain:

- Automation: Executes actions automatically when conditions are satisfied.
- Transparency: The contract code and transactions are visible on the blockchain.
- Security: Once deployed, contracts are immutable and tamper-proof.
- Cost-effective: Removes intermediaries and reduces transaction costs.
- Trustless System: Participants can interact without needing to trust each other directly.

Smart contracts form the backbone of decentralized applications (DApps) and decentralized finance (DeFi) systems on Ethereum.

Code:\

```
// SPDX-License-Identifier: MIT pragma  
solidity ^0.8.0;
```

```
contract PharmaSupply {
```

```
    address public manufacturer;  
    address public pharmacy;
```

```
    struct Medicine {  
        string name;  
        uint quantity;  
        bool shipped;  
        bool received;  
    }
```

```
    Medicine public med;
```

```
    constructor(address _pharmacy) {  
        manufacturer = msg.sender;  
        pharmacy = _pharmacy;
```

```

}

function addMedicine(string memory _name, uint _quantity) public {
    require(msg.sender == manufacturer, "Only manufacturer allowed"); med
    = Medicine(_name, _quantity, false, false);
}

function shipMedicine() public {
    require(msg.sender == manufacturer, "Only manufacturer allowed");
    med.shipped = true;
}

function receiveMedicine() public {
    require(msg.sender == pharmacy, "Only pharmacy allowed");
    require(med.shipped == true, "Not shipped yet");
    med.received = true;
}

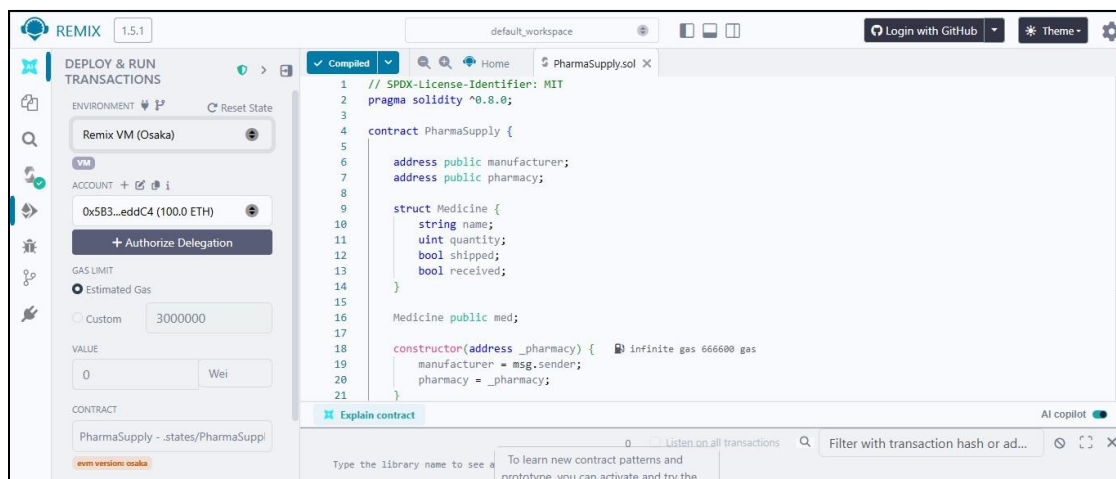
function getMedicine() public view returns (string memory, uint, bool, bool) { return
    (med.name, med.quantity, med.shipped, med.received);
}
}

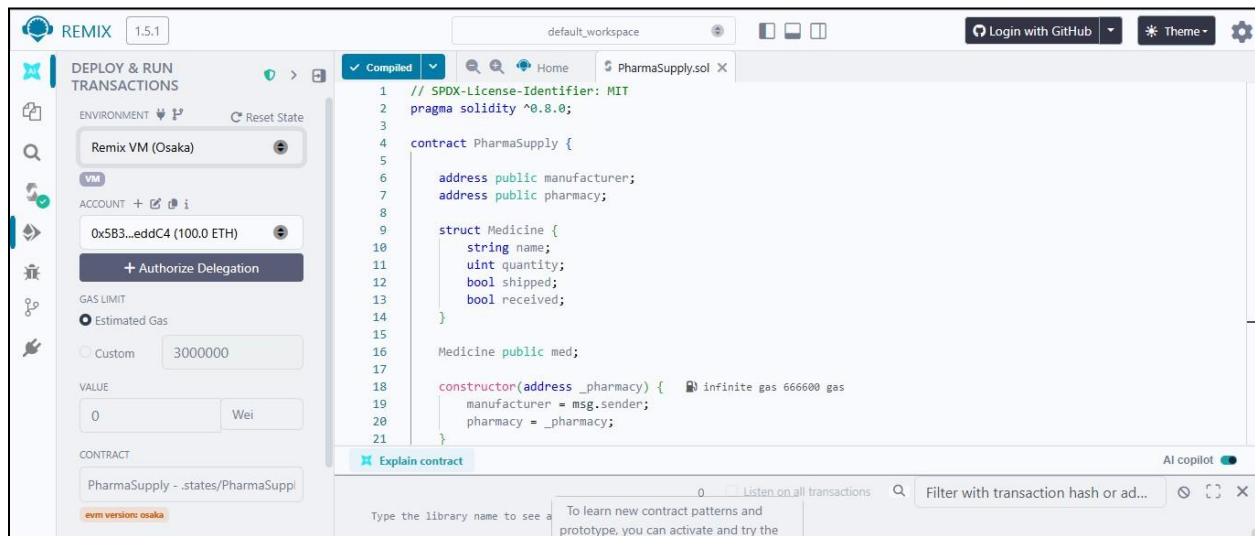
```

Implementation:

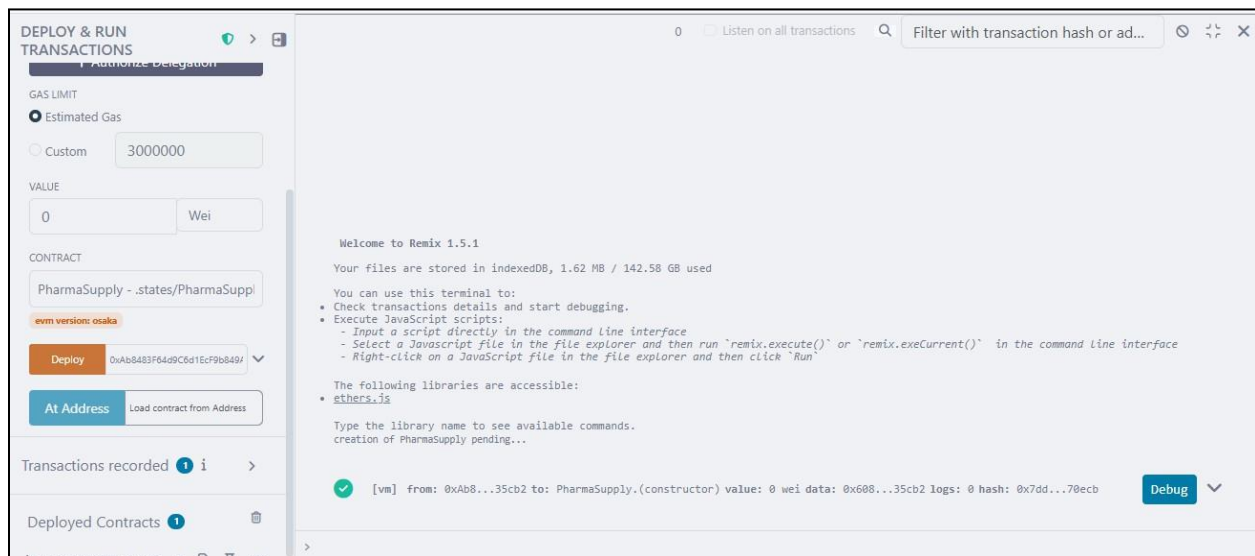
Contract 1: PharmaSupply (Medicine flow)

Contract code





Transaction deployed successfully



Deployed contract

DEPLOY & RUN TRANSACTIONS

☒ Estimated Gas

☐ Custom 3000000

VALUE

0 Wei

CONTRACT

PharmaSupply - .states/PharmaSupply.sol

evm version: osaka

Deploy 0xAb8483F64d9C6d1EcF9b849f

At Address Load contract from Address

Transactions recorded 1

Deployed Contracts 1

PHARMASUPPLY AT 0XA13...EA

Manufacture adding medicine for shipping

DEPLOY & RUN TRANSACTIONS

Deployed Contracts

PHARMASUPPLY AT 0XA13...EA

Balance: 0 ETH

ADD MEDICINE

_name: Paracetamol

_quantity: 100

Calldata Parameters transact

receiveMedicine

shipMedicine

getMedicine

manufacturer

med

Compile

Home PharmaSupply.sol Solidity Compile Details

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract PharmaSupply {
5
6     address public manufacturer;
7     address public pharmacy;
8
9     struct Medicine {
10         string name;
11         uint quantity;
12         bool shipped;
13         bool received;
14     }
15
16     Medicine public med;
17
18     constructor(address _pharmacy) { infinite gas 666600 gas
```

Explain contract

0 Listen on all transactions

Welcome to Remix 1.5.1

Your files are stored in indexedDB, 1.62 MB / 142.58 GB used

Manufacturer successfully added medicine

The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. On the left, the 'ADD MEDICINE' form is visible with the following details:

- Balance: 0 ETH
- ._name: "Paracetamol"
- ._quantity: 100
- Buttons: Calldata, Parameters, **transact**, receiveMedicine, shipMedicine, getMedicine, manufacturer, med, pharmacy.

The transaction logs on the right show the following sequence of events:

- Reverted Transaction 1:** [vm] from: 0x583...eddC4 to: PharmaSupply.addMedicine(string,uint256) 0xa13...eAD95 value: 0 wei data: 0xa98...00000. Logs: 0 hash: 0xf9a...47e9a. Reason: "Only manufacturer allowed".
- Reverted Transaction 2:** [vm] from: 0x583...eddC4 to: PharmaSupply.addMedicine(string,uint256) 0xa13...eAD95 value: 0 wei data: 0xa98...00000. Logs: 0 hash: 0x954...3fd40. Reason: "Only manufacturer allowed".
- Successful Transaction 3:** [vm] from: 0xAb8...35cb2 to: PharmaSupply.addMedicine(string,uint256) 0xa13...eAD95 value: 0 wei data: 0xa98...00000. Logs: 0 hash: 0xb32...a5da8.
- Successful Transaction 4:** [vm] from: 0xAb8...35cb2 to: PharmaSupply.addMedicine(string,uint256) 0xa13...eAD95 value: 0 wei data: 0xa98...00000. Logs: 0 hash: 0x372...ebb36.

Manufacturer Shipping the medicine

The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. On the left, the 'SHIP MEDICINE' form is visible with the following details:

- ._name: "Paracetamol"
- ._quantity: 100
- Buttons: Calldata, Parameters, **transact**, receiveMedicine, shipMedicine, getMedicine, manufacturer, med, pharmacy.

The transaction logs on the right show the following sequence of events:

- Call Transaction:** [call] from: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2 to: PharmaSupply.manufacturer() data: 0x747...54282. Execution cost: 2552 gas. Decoded output: {"@": "address: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2"}.
- Successful Transaction:** [vm] from: 0xAb8...35cb2 to: PharmaSupply.shipMedicine() 0xa13...eAD95 value: 0 wei data: 0x39f...32075 logs: 0 hash: 0xe26...bd946.

Medicine received by pharmacy successfully

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the contract's balance as 0 ETH and a list of functions: addMedicine, receiveMedicine, shipMedicine, getMedicine, manufacturer, med, and pharmacy. The 'receiveMedicine' function is selected. The main panel shows the transaction logs. The first two transactions are marked with a red 'x' and indicate a revert error: 'Error occurred: revert.' The reason provided by the contract is 'Only pharmacy allowed'. The third transaction is marked with a green checkmark and indicates a successful transaction: 'transaction to PharmaSupply.receiveMedicine pending ...'. The fourth transaction is also marked with a green checkmark and indicates a successful transaction: 'transaction to PharmaSupply.receiveMedicine pending ...'.

getmedicine() we are getting the values of added medicine

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the contract's balance as 0 ETH and a list of functions: addMedicine, receiveMedicine, shipMedicine, getMedicine, manufacturer, med, and pharmacy. The 'getMedicine' function is selected. The main panel shows the output of the function call. The output is a list of values: 0: string: Paracetamol, 1: uint256: 100, 2: bool: true, 3: bool: true. Below the output, the 'manufacturer' function is selected, and its output is displayed: 0: address: 0xAb8483F64d9C6d1EcF9b849Ae677dD3315835cb2.

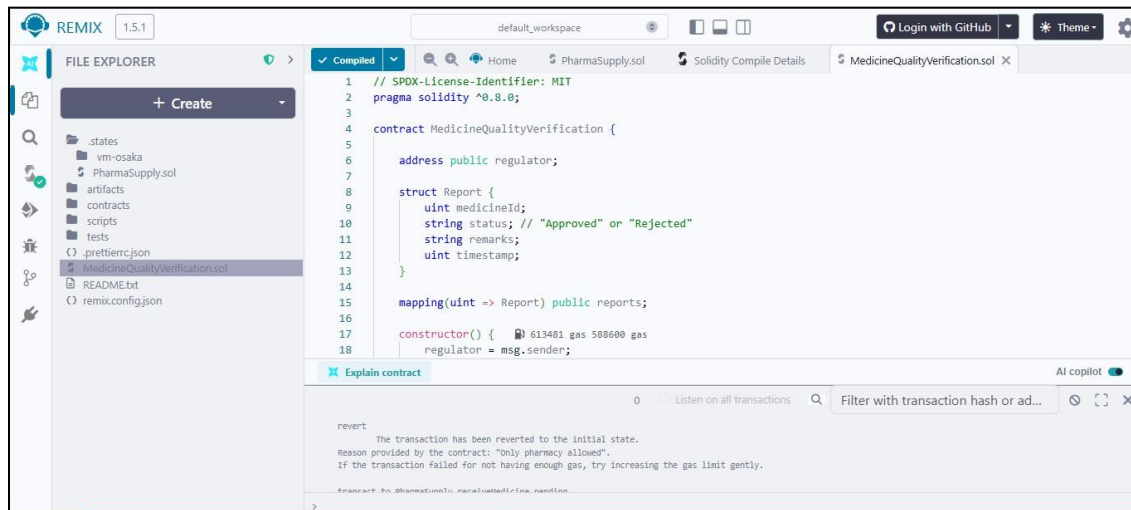
Contract 2: medicine quality verification

While verifying:

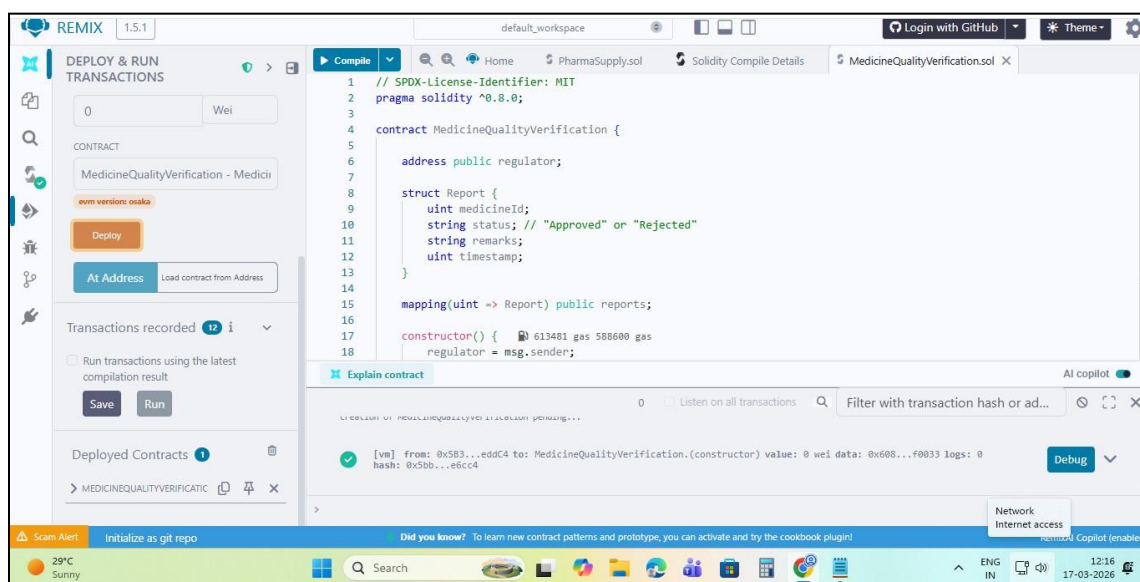
“Here, the regulator verifies the medicine batch and stores its approval status on blockchain.”

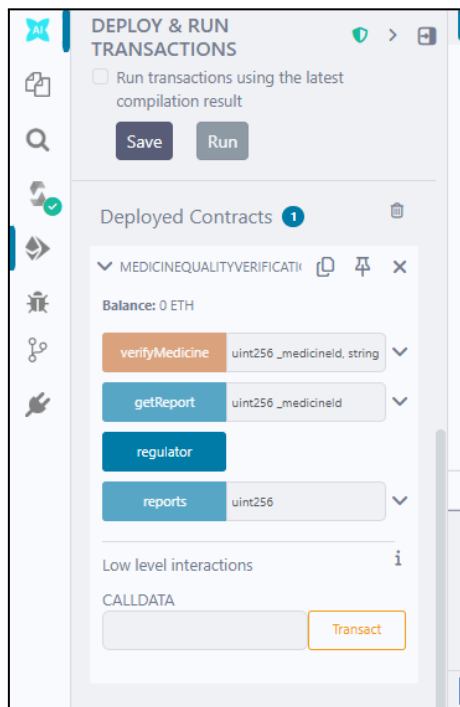
While fetching:

“This function retrieves the verification report, ensuring transparency and immutability.”

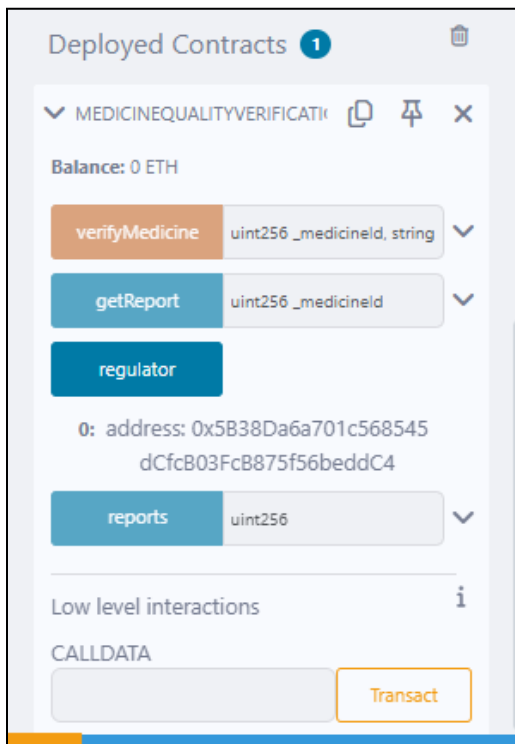


Contract deployed

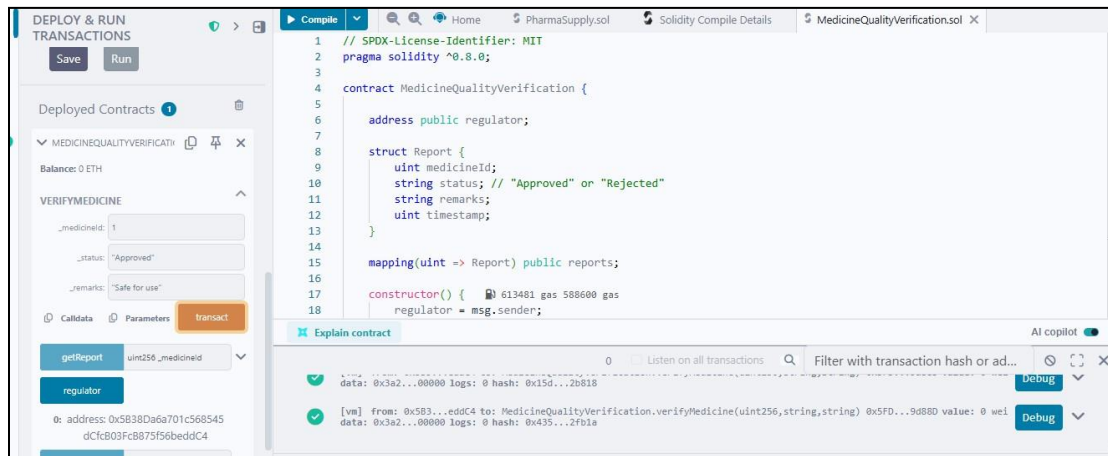




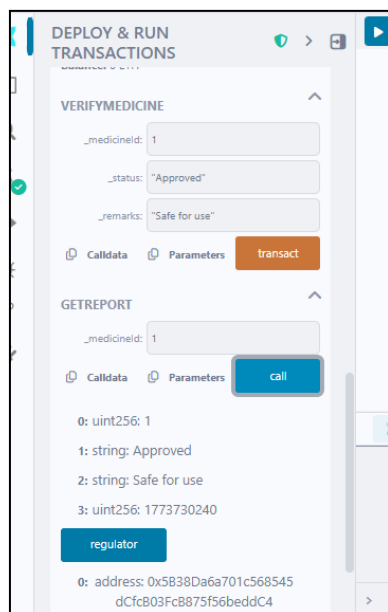
Regulator



Medicine verification done by regulator



getreport



Conclusion:

The experiment was successfully completed using Remix IDE, where two Solidity smart contracts were deployed and tested on the Ethereum blockchain. Contract 1 (PharmaSupply) demonstrated the medicine flow from manufacturer to pharmacy, while Contract 2 (MedicineQualityVerification) allowed a regulator to verify and store quality reports on-chain. Role-based access control was enforced using `require` statements, and all transactions were recorded immutably. The experiment successfully demonstrated the use of smart contracts for transparent and trustless pharmaceutical supply chain management.